DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1 (CO1)

Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Submitted by: G. Sirisha

Register Number: 192472170

Subject: DSA03 – Natural Language Processing

Faculty Name: Premkumar R

Submission Date: 04-11-26

ABSTRACT

This assignment designs a finite-state text-processing layer for a hospital record search system. The proposed pipeline recognises structured clinical entities, tokenises and normalises free text, validates formats with finite automata, analyses English inflection and derivation with a finite-state transducer (FST), and compares dictionary-based lemmatisation with the Porter stemmer. A Day-4 redesign adds OCR-tolerant matching and support for Romanised regional-language/English code-mixing.

The report uses a documented synthetic evaluation design rather than claiming access to real patient records. The planned corpus contains 500 synthetic, de-identified records, 25,600 word tokens and four record types: discharge summaries, prescriptions, operation notes and nursing observations. All example identifiers and values are fictional. Numerical evaluation figures labelled “illustrative pilot” are internally consistent worked results intended to demonstrate how the experiment should be reported; they must be replaced with actual execution results if the assignment is submitted as an empirical implementation.

PROJECT SCOPE AND DATASET

The system is a retrieval-support component, not a diagnostic tool. It must preserve the original text, store normalised text separately, and never silently change a clinical value. A match is evidence that a string has a recognised form, not proof that the clinical statement is true.

- Corpus plan: 500 synthetic records; 125 discharge summaries, 125 prescriptions, 125 operation notes and 125 nursing observations.
- Corpus size: 25,600 whitespace-delimited word tokens after record segmentation; average 51.2 tokens per record.
- Lexicon plan: 320 stems, 24 inflectional affixes, 18 derivational affixes, and a separate irregular-form list.
- Evaluation: 40 clinician-style queries with graded relevance; 200 manually annotated word forms; 12 entity patterns; four automata.
- Privacy: no real names, phone numbers, patient identifiers or actual clinical records are used. Identifiers in examples are synthetic.

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS (3 MARKS)

1.1 Levels of linguistic analysis

Phonology studies sound systems and pronunciation. It is not central to the clean typed-English baseline because the records are written text. It becomes relevant indirectly in OCR and Romanised language because spelling may reflect pronunciation, for example a regional-language form typed as “irundhuchu”.

Morphology studies the internal structure of words and how stems combine with affixes. It is essential: fracture, fractures, fractured and fracturing should be connected through the lemma fracture; nephrectomy and nephrectomies share a stem and plural feature. Derivation also matters, but must be treated cautiously: renal and nephrectomy are clinically related, not morphological variants of the same lemma.

Syntax studies how words combine into phrases and sentences. It is useful for interpreting dosage phrases such as “take one tablet twice daily” and for distinguishing “no evidence of fracture” from “evidence of fracture”. The first finite-state layer only needs shallow phrase boundaries and negation cues; full parsing is outside the initial scope.

Semantics concerns meaning. It is needed for clinically useful query expansion and synonym handling, such as linking renal with kidney, but a morphology-only system cannot infer all such relationships. A curated synonym map is safer than pretending that derivation provides complete semantic equivalence.

Pragmatics studies meaning in context, including speaker intention and situational use. It is not required for exact entity validation. It may matter when interpreting shorthand such as “review tomorrow” or “hold drug”, so it is deferred to a later clinical-context layer.

Discourse studies relations across sentences and documents, such as whether a later sentence corrects an earlier statement. It is not necessary for token normalisation, but it is important for resolving temporal progression and contradictory notes. The proposed search layer preserves document and section boundaries so a future discourse component can use them.

1.2 Model families

State machines are appropriate for validating regular formats: a patient identifier such as HSP-2026-00418 or a date in YYYY-MM-DD form. A finite automaton gives explicit accepted and rejected paths. A probabilistic model is unnecessary for exact format validation because it could assign a high score to an invalid identifier. A large rule system is harder to audit for this finite pattern.

Formal rule systems are appropriate for English inflection and orthographic alternations: plural -s, -es, y-to-i, and past-tense spelling. A rule system is deterministic, inspectable and can explain an analysis. A state machine alone does not conveniently encode a productive lexicon and feature structure; a probabilistic model may produce plausible but unsupported forms.

Probabilistic models are appropriate for ranking uncertain OCR corrections or choosing between ambiguous analyses using context. They are not the first choice for exact date validation because a likely date is not necessarily a valid date, and they are not needed for deterministic spelling rules.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “lead” can mean a metal, a clinical ECG lead, or the verb “lead”. The surrounding phrase is needed.
- Morphological ambiguity: “draining” may be a verb form of drain or an adjective in “draining wound”.
- Segmentation ambiguity: “12lead ECG” may need to be segmented as “12-lead ECG”; “500mg” should be recognised as a numeric value plus unit, not one ordinary word.

- Abbreviation ambiguity: “OD” may mean once daily in a prescription, right eye in ophthalmology, or something else in another section. Section and context are essential.

1.4 Search and resource costs

Automaton recognition explores states while consuming input. A deterministic automaton takes O(n) time and O(1) working memory for a string of length n, apart from the stored transition table. NFA simulation can maintain a set of active states and costs approximately O(n|Q|) in a simple implementation. Backtracking regular-expression engines may revisit alternatives; poorly designed nested quantifiers can cause exponential worst-case time. The proposed patterns avoid nested ambiguous repetition and use anchors.

The main costs are: time for scanning records and queries; memory for the lexicon, transition tables and inverted index; annotated data for relevance and gold-standard morphology; and expert effort to define safe clinical abbreviations, synonyms and exceptions. Annotation and clinical review are the most expensive human resources.

1.5 Proposed pipeline diagram

INPUT: discharge summaries / prescriptions / operation notes / nursing notes
↓ [UTF-8 decoding, record metadata]
OCR/encoding cleanup (when required) — character rules + confidence scores
↓
TOKENISATION — regular expression + maximal-munch scanner
↓
STRUCTURED ENTITY RECOGNITION — anchored regex + DFA validators
↓
NORMALISATION — case folding, abbreviation map, date/unit canonicalisation
↓
MORPHOLOGICAL ANALYSIS — lexicon FST + continuation classes + spelling rules
↓
FALLBACK — Porter stemmer for unknown English-like tokens; retain original token
↓
INDEXING — lemma, features, entity fields, original text and section metadata
↓
OUTPUT: ranked searchable records + highlighted matched spans + provenance

Every stage stores offsets back to the original text. This is important because a clinician must be able to inspect the exact source phrase even when the index uses a canonical form.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS (4 MARKS)

2.1 Basic operators and clinical examples

Operator	Meaning	Minimal example
Concatenation	A followed by B	HSP followed by digits: HSP[0-9]+
Disjunction |	Either alternative	mg|mcg
Kleene star *	Zero or more repetitions	[0-9]*
Plus +	One or more repetitions	[0-9]+
Optional ?	Zero or one occurrence	-? for optional minus sign
Character class	One character from a set	[ABC]
Range	Characters within a range	[0-9], [A-Z]
Negated class	Any character except listed set	[^\s,;]+
Anchors	Start/end of string	^HSP-[0-9]{4}-[0-9]{5}$
Counters {m,n}	Bounded repetition	[0-9]{2,4}

2.2 Twelve-entity pattern inventory

The following patterns are written in Python regular-expression syntax. For production use, each pattern is applied to a field or candidate span and validated with fullmatch where appropriate. The patterns recognise formats; they do not certify that the value is clinically correct.

1. Patient identifier

```
\\bHSP-[0-9]{4}-[0-9]{5}\\b
```

Must match HSP-2026-00418; must reject HSP-26-418 and embedded partial strings.

2. Admission date

```
\\b(?:[0-9]{4}-[0-9]{2}-[0-9]{2}|[0-9]{2}/[0-9]{2}/[0-9]{4})\\b
```

Must match ISO or DD/MM/YYYY candidate dates; must reject 2026/31/12. Calendar validity is checked after regex.

3. Discharge date

```
\\b(?:[0-9]{4}-[0-9]{2}-[0-9]{2}|[0-9]{2}/[0-9]{2}/[0-9]{4})\\b
```

Same format as admission date; semantic field label distinguishes discharge from admission.

4. Drug name + strength

```
\\b[A-Za-z][A-Za-z0-9-]*(?:\\s+\\d+(?:\\.\\d+)?\\s*(?:mg|mcg|g|ml|mL|IU))\\b
```

Must match paracetamol 500mg and amoxicillin 250 mg; must reject a bare drug name or unsupported unit.

5. Dosage frequency

```
\\b(?:OD|BD|TDS|QID|once daily|twice daily|three times daily|every\\s+\\d+\\s+hours)\\b
```

Must match recognised frequency expressions; must reject arbitrary strings such as daily-ish.

6. Laboratory value + unit

```
\\b(?:Hb|WBC|RBC|glucose|creatinine|sodium|potassium)\\s*[:=]?\\s*[0-9]+(?:\\.[0-9]+)?\\s*(?:g/dL|10\\^9/L|mmol/L|mg/dL|mEq/L)\\b
```

Must match Hb 12.4 g/dL; must reject missing units or alphabetic values.

7. Blood pressure

```
\\b(?:BP\\s*)?[0-9]{2,3}\\s*/\\s*[0-9]{2,3}\\s*mmHg\\b
```

Must match BP 120/80 mmHg; must reject 120/80 without unit in strict mode.

8. Temperature

```
\\b(?:Temp\\s*)?[0-9]{2}(?:\\.[0-9])?\\s*°?\\s*[CF]\\b
```

Must match Temp 37.2 C or 98.6°F; must reject 370 C.

9. ICD code

```
\\b[A-TV-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b
```

Must match illustrative ICD-like code S72.001A; must reject strings with invalid leading characters.

10. Ward and bed

```
\\bWard\\s+[A-Z0-9-]+\\s*,?\\s*Bed\\s+[A-Z0-9-]+\\b
```

Must match Ward B2, Bed 14; must reject incomplete ward-only strings.

11. Contact number

\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b

Must match a ten-digit Indian mobile-style number; must reject nine or eleven digits.

12. Age

\\b(?:Age\\s*[:=]?\\s*)?(?:0|[1-9][0-9]?|1[01][0-9]|120)\\s*(?:years?|yrs?)\\b

Must match Age: 42 years; must reject 145 years.

2.3 Greedy and non-greedy matching

Sentence: “Administer 500 mg paracetamol after food; repeat 250 mg paracetamol if prescribed.” Consider the deliberately broad expression `paracetamol.*mg` versus `paracetamol.*?mg`. Greedy `.*` consumes as much as possible while still allowing the final `mg` to match, so it can span from the first “paracetamol” to the later “mg” after 250. Non-greedy `.*?` expands only as far as needed and stops at the first following “mg”. For extraction, a bounded dosage pattern is safer than either broad expression.

2.4 Precedence and grouping

Concatenation binds more tightly than disjunction. The expression `A|BC` denotes either A or the two-character sequence BC. If the intention is “A or B, followed by C”, the correct expression is `(A|B)C`. For example, `OD|BDdaily` matches `OD` or `BDdaily`, not `ODdaily` or `BDdaily`. The corrected `(OD|BD)daily` matches the intended alternatives. Grouping prevents silent language changes.

2.5 Evaluation and error causes

Illustrative pilot evaluation on 1,200 manually labelled entity spans from the synthetic corpus is summarised below. Counts are examples of a reporting format and should be replaced by a run on the final corpus. TP means correctly detected spans; FP means a false detection; FN means a missed gold span.

Entity family	**TP**	**FP**	**FN**	**Main error cause**
Identifiers	98	1	2	OCR-like O/0 confusion in one

				candidate
Dates	176	8	7	Ambiguous DD/MM versus MM/DD; invalid calendar day
Drug + strength	142	11	13	Brand spelling and whitespace variation
Frequency	91	9	12	Context-dependent OD/BD abbreviations
Lab value + unit	128	7	10	Unit variants and missing spaces
BP / temperature	105	4	6	Unit omitted or unusual notation
ICD / ward-bed / age / contact / duration	224	13	15	Local formatting and incomplete labels

False positives arise mainly from broad lexical alternatives, ambiguous abbreviations and values that look structurally valid but are in the wrong context. False negatives arise from spelling variants, missing punctuation, unrecognised units and inconsistent separators. Regex should therefore be followed by calendar/range validation and section-aware checks.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION (4 MARKS)

3.1 Token definition and tokenizer

A token is the longest contiguous item in one of these classes: a decimal or integer number; a number immediately followed by a recognised unit; an abbreviation with internal full stops; a hyphenated alphabetic term; an alphanumeric clinical code; or an ordinary word with an optional apostrophe.

Punctuation is emitted separately only when it carries meaning, such as a decimal point inside a number or a slash inside blood pressure.

```
(?ix)(?:[0-9]+(?:\.[0-9]+)?\s*(?:mg|mcg|g|ml|mL|mmol/L|mg/dL|mmHg|IU))|(?:[A-Za-z](?:\.[A-Za-z])+\.?)|(?:[A-Za-z]+(?:-[A-Za-z]+)+)|(?:[A-Za-z0-9]+(?:['’][A-Za-z]+)?)|(?:[0-9]+(?:\.[0-9]+)?)
```

The scanner tests alternatives in priority order and selects the longest match at the current position (maximal munch). Units are placed before the generic number alternative so “500mg” is one dosage token. In the index, it may additionally be represented as numeric value 500 plus unit mg.

3.2 Maximal-munch examples

- “BP 120/80 mmHg” – whitespace splitting gives BP, 120/80, mmHg; the clinical scanner recognises BP, 120/80, mmHg as separate structured components and can combine them into one BP entity.
- “12-lead ECG” – whitespace splitting gives 12-lead and ECG, but a naive punctuation split may produce 12, lead, ECG. The maximal-munch rule preserves 12-lead as one token.
- “Hb 12.4g/dL” – whitespace splitting gives Hb and 12.4g/dL; the scanner preserves the decimal and unit boundary and records the numeric value and unit.
- “i.v. fluids” – a naive split can split the abbreviation at periods; the abbreviation alternative preserves i.v. as one token.

3.3 Substitution and canonicalisation rules

Case folding uses Unicode casefolding rather than a simple ASCII lowercase operation. A curated abbreviation dictionary is applied only when the section and local context support the expansion. The following 30 mappings form the initial inventory:

BP → blood pressure; HR → heart rate; RR → respiratory rate; Temp → temperature; Hb → haemoglobin; WBC → white blood cell count; RBC → red blood cell count; ECG → electrocardiogram; CXR → chest X-ray; CT → computed tomography; MRI → magnetic resonance imaging; USG → ultrasonography; IV → intravenous; IM → intramuscular; PO → by mouth; PRN → as needed; OD → once daily (prescription context); BD → twice daily; TDS → three times daily; QID → four times daily; SOS → if required; NPO → nothing by mouth; Dx → diagnosis; Hx → history; Rx → prescription/treatment; F/U → follow-up; c/o → complains of; h/o → history of; s/p → status post; WNL → within normal limits.

Date canonicalisation converts validated dates to YYYY-MM-DD while retaining the original string and the detected locale assumption. Unit canonicalisation maps spelling variants to a controlled unit code, for example “milligrams”, “mg” → mg. It does not convert values unless a dimensionally valid conversion is explicitly implemented.

3.4 Full-record before and after example

Synthetic input record:

> Patient HSP-2026-00418 admitted on 04/11/2026. C/O fever x 2 days. BP 120/80 mmHg, Temp 37.2 C. Hb 12.4 g/dL. Rx: Paracetamol 500mg PO BD. F/U after 3 days.

Normalised representation:

> patient hsp-2026-00418 admitted on 2026-11-04. complains of fever duration 2 days. blood pressure 120/80 mmhg, temperature 37.2 c. haemoglobin 12.4 g/dl. prescription: paracetamol 500 mg by mouth twice daily. follow-up after 3 days.

The original record remains unchanged in storage. Normalised text is an additional search field, with character offsets and transformation logs.

3.5 Irreversible decisions and limits

- Case folding loses original capitalisation. This is acceptable for ordinary search but not when distinguishing a code or an acronym whose case is meaningful; original text is retained.
- Abbreviation expansion loses the exact shorthand used by the clinician. It is acceptable for retrieval when the abbreviation is unambiguous in context; otherwise both original and expanded forms are indexed.
- Date canonicalisation can lose the original locale interpretation. It is unsafe for ambiguous values such as 04/05/2026 without a known date convention; such values are flagged rather than silently converted.
- Removing punctuation can destroy decimal, code or negation boundaries. Punctuation is therefore retained when it affects clinical meaning.

A regular expression cannot recognise arbitrarily nested balanced parentheses, such as a requirement that every opening parenthesis has a matching closing parenthesis at any depth. The language of balanced parentheses is not regular; it requires unbounded memory such as a stack. Regex engines with recursion extensions may support it, but that goes beyond classical regular expressions and finite automata.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE (5 MARKS)

4.1 Formal definition

A finite automaton is a five-tuple M = (Q, Σ, δ, q0, F), where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q0 is the start state and F is the set of accepting states. A DFA has exactly one next state for every state-symbol pair (with a dead state for missing transitions).

4.2 Four DFA validators

The validators below are intentionally described over character classes to keep the transition tables readable. Each DFA has at least six logical states. In an implementation, transitions on a character class are expanded or represented by a class predicate.

DFA A: Patient identifier HSP-YYYY-NNNNN

States: q0 start, q1 after H, q2 after HS, q3 after HSP-, q4 after 4 year digits, q5 after separator, q6 after 1–4 serial digits, q7 accept after 5 serial digits, qD dead. Diagram: q0 -H→ q1 -S→ q2 -P→ q3 --→ q4; q4 -digit×4→ q5; q5 --→ q6; q6 -digit×5→ q7(accept). Any wrong character → qD.

State	Input class	Next state	Meaning
q0	H	q1	Prefix H
q1	S	q2	Prefix HS
q2	P	q3	Prefix HSP
q3	-	q4	Separator
q4	digit	q4a	Year digit 1
q4a	digit	q4b	Year digit 2
q4b	digit	q4c	Year digit 3
q4c	digit	q5	Year complete
q5	-	q6	Serial begins
q6	digit×1–4	q6	Serial partial
q6	digit (5th)	q7	Accept
any	other	qD	Reject

DFA B: ISO date YYYY-MM-DD

States: d0 start; d1–d4 consume four year digits; d5 consume hyphen; d6–d7 month digits; d8 second hyphen; d9–d10 day digits; d11 accept; dD dead. Calendar range checks (month 01–12 and day valid for month/year) are applied after structural acceptance.

State group	Allowed input	Next	Notes
d0–d3	digit	next year state	Exactly four year digits
d4	-	d5	Year separator
d5	0 or 1	d6	Month tens
d6	digit with valid month range	d7	Month completed
d7	-	d8	Day separator
d8	0–3	d9	Day tens
d9	digit	d10	Day completed
d10	end of input	d11	Accept structure
any	invalid input	dD	Reject

DFA C: Blood pressure systolic/diastolic mmHg

States: b0 start; b1–b3 systolic digits (2 or 3); b4 slash; b5–b7 diastolic digits (2 or 3); b8 optional space; b9–b12 literal mmHg; b13 accept; bD dead. Range checks such as systolic 50–260 and diastolic 30–160 are post-validation rules.

DFA D: Dosage frequency

States: f0 start; f1 sees O; f2 sees OD (accept); f3 sees B; f4 sees BD (accept); f5 sees T; f6 sees TD; f7 sees TDS (accept); f8 sees Q; f9 sees QI; f10 sees QID (accept); fD dead. A trie-shaped DFA shares prefixes and rejects incomplete abbreviations.

4.3 NFA with epsilon transitions: optional date prefix

For a date label that may be absent, define the language `(?:Date:\s*)?YYYY-MM-DD`. The NFA branches from the start state using ε: one branch skips the label and enters the year parser; the other consumes “Date:” and optional spaces before entering the same year parser. This is convenient because the optional prefix creates two possible paths.

NFA states: n0 start; n1 after Date:; n2 after optional spaces; n3 year; n4 hyphen; n5 month; n6 hyphen; n7 day; n8 accept.
ε transitions: n0→n3 (no label), n0→n1 (label path), n1→n2, n2→n3.
For input Date: 2026-11-04, active states progress through label path to n3, then year/hyphen/month/hyphen/day to n8. For input Date: 2026-13-40, the structural NFA may reach accept; a calendar validator rejects month 13 and day 40.

4.4 Subset construction and minimisation

Subset construction creates DFA states that are sets of NFA states. The ε-closure of a set is computed before and after consuming each symbol. For the simplified optional-prefix NFA above, the start subset is ε-closure({n0}) = {n0,n1,n3}. After reading the first symbol, only paths compatible with that symbol remain. Equivalent subsets are merged during DFA minimisation.

DFA subset	Input class	Next subset	Interpretation
{n0,n1,n3}	D	{n1}	Label path begins
{n0,n1,n3}	digit	{n3}	Unlabelled date begins
{n1}	a	{n1a}	Continue label
{n3}	digit	{n3a}	Year digit
{n3a}	digit	{n3b}	Year digit
{n3b}	digit	{n3c}	Year digit
{n3c}	-	{n4}	Year complete
{n4}	digit	{n5}	Month begins
{n5}	-	{n6}	Month complete
{n6}	digit	{n7}	Day begins
{n7}	end	{n8}	Accept

For a complete implementation, all character classes and dead transitions are included. In this simplified construction, 12 reachable subsets reduce to 10 equivalence classes after merging the two paths that enter the common date parser. The exact count depends on whether the literal label is modelled character-by-character or as a token transition; the report should state that representation.

4.5 Thompson construction and reverse derivation

For the expression `a(b|c)*`, Thompson construction creates an NFA fragment for `a`, an ε-branch for b or c, and an ε-loop for zero or more repetitions. The start consumes a, then the loop permits any number of b/c symbols, including zero. The accepted strings are a, ab, ac, abb, abc, and so on. Conversely, eliminating states from a two-branch automaton with an initial a transition and a b/c loop yields `a(b|c)*`. Each expression-generated path corresponds to an automaton path, and each accepted path spells a string in the expression language.

4.6 Pumping lemma and non-regularity

Pumping lemma: if L is regular, there is a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, with $|xy| \leq p$, $|y| > 0$, and $xy^i z \in L$ for every $i \geq 0$.

Let L = { `(`^n `)`^n : n ≥ 0 }, the strings with equal numbers of opening and closing parentheses in one nested run. Assume L is regular and let p be its pumping length. Choose s = `(`^p `)`^p. Since $|xy| \leq p$, y contains only opening parentheses and has length greater than zero. Pumping down with i=0 gives fewer opening parentheses than closing parentheses, so xz is not in L. This contradicts the lemma; therefore L is not regular. A classical finite automaton cannot validate arbitrarily nested balanced parentheses.

TASK 5 – INFLECTIONAL MORPHOLOGY (3 MARKS)

5.1 Core definitions

A morpheme is the smallest unit that carries grammatical or lexical meaning. A stem is the form to which an inflectional or derivational affix attaches. An affix is a bound morpheme attached before or after a stem. Inflection changes grammatical features while normally preserving word class and core lexical identity. A paradigm is the set of forms associated with one lexeme, such as singular/plural or present/past.

Inflection differs from derivation because it generally preserves category, is more productive and occurs outside derivational morphology. For example, nephrectomies can be analysed as nephrectomy + plural; -ies expresses number. By contrast, nephrectomy is a derived medical noun built with -ectomy.

5.2 Inflectional paradigms in the corpus

The corpus contains noun number (singular/plural), verb tense (present/past), progressive aspect (-ing), third-person singular agreement (-s), and adjective comparison (-er/-est). Clinical notes are

telegraphic, so not every paradigm occurs equally often. Agreement is less frequent than noun plural and participles.

fracture: fracture, fractures, fractured, fracturing.

treat: treat, treats, treated, treating.

infect: infect, infects, infected, infecting.

wound: wound, wounds, wounded, wounding.

nephrectomy: nephrectomy, nephrectomies.

5.3 Implementable spelling rules

- E-deletion: delete final silent e before a vowel-initial suffix: fracture + ing → fracturing; dose + ing → dosing. Counter-example: dye + ing → dyeing (deletion would create "dying", a different word).
- Consonant doubling: double a final consonant after a stressed short vowel in a monosyllabic or appropriately stressed final syllable before a vowel-initial suffix: stop + ed → stopped. Counter-example: visit + ed → visited, not visitted.
- Y-to-i: consonant + y changes y to i before -es or -ed: biopsy + es → biopsies; carry + ed → carried. Counter-example: play + ed → played because y follows a vowel.
- Sibilant -es: add -es after s, x, z, ch, sh: patch + es → patches; dose + s → doses. Counter-example: photo + s → photos; a blanket -es rule would over-generate.

5.4 Irregular forms

The irregular list is separate because the form cannot be predicted by the regular suffix rules alone: be → am/is/are/was/were/been; have → has/had; do → does/did/done; go → went/gone; child → children; foot → feet; tooth → teeth; person → people; mouse → mice; diagnosis → diagnoses; analysis → analyses. Some clinical vocabulary also contains historically irregular plural forms such as bacterium → bacteria and fungus → fungi. A lexicon stores these mappings with grammatical features.

5.5 Measurement

Illustrative pilot: 3,840 distinct surface word types map to 2,760 lemmas, giving a type-to-lemma ratio of 3,840 / 2,760 = 1.39. In the 40-query evaluation, 52 of 160 judged missed relevant matches (32.5%) were attributable solely to inflectional mismatch, such as searching "fracture" while a record

contains “fractures”. This is a retrieval-oriented measure, not a claim that every type is morphologically distinct.

TASK 6 – DERIVATIONAL MORPHOLOGY (4 MARKS)

6.1 Derivational affix inventory

Derivation creates a new lexeme and may change word class or meaning. The following affixes are included in the synthetic clinical lexicon. Prefixes are shown with a hyphen after them; suffixes with a hyphen before them.

Affix	**Typical contribution / category**	**Example**
-itis	noun: inflammation	arthritis
-ectomy	noun: surgical removal	nephrectomy
-ology	noun: field of study	cardiology
-osis	noun: condition/state	fibrosis
-aemia	noun: blood condition	anaemia
-ation	noun from verb	inflammation
-ment	noun from verb	treatment
-able	adjective from verb	treatable
-ive	adjective	protective
-al	adjective	renal
-ic	adjective	cardiac
-ist	person noun	specialist
re-	repeat/action again	reoperation
un-	negation/reversal	unhealed
hyper-	above/excessive	hypertension
hypo-	below/deficient	hypoglycaemia
nephro-	kidney-related combining form	nephrology
cardio-	heart-related combining form	cardiovascular

6.2 Derivational families and ordering

Family 1 – infect: infect (V) → infection (N, -ion) → infectious (Adj, -ous) → non-infectious (Adj, non- + infectious). The prefix non- attaches to an already formed adjective in this example.

Family 2 – operate: operate (V) → operation (N, -ation) → operational (Adj, -al) → preoperative (Adj, pre- + operative).

Family 3 – renal/kidney: nephro- + logy → nephrology (N); nephro- + -ectomy → nephrectomy (N). These are related through a combining form, not a single inflectional paradigm.

Family 4 – cardiac: cardio- + logy → cardiology (N); cardiology + -ist → cardiologist (N, person); cardiac (Adj, -ac).

Family 5 – treat: treat (V) → treatment (N, -ment) → treatable (Adj, -able) → untreatable (Adj, un- + treatable).

Family 6 – fracture: fracture (N/V) → fractured (participle/adjective, inflectional in one use) → refracture (N/V, re- derivation). The same ending may be inflectional or lexicalised depending on use.

Impossible or ill-formed orders include `-ectomy + nephro-` after the suffix (the combining form must precede the head); and `-able + un- + treat` if interpreted as a suffix attaching before the stem. The correct order is un- + treat + -able.

6.3 Non-compositionality and productivity

A derived form is not always semantically predictable from its pieces. “Inflammation” is not merely any action of inflaming; it names a clinical process or state. “Operation” has a broad everyday meaning and a specialised surgical meaning. “Discharge” can mean release from hospital, fluid from a wound, or an electrical event. Therefore aggressive stemming can collapse distinct clinical senses.

Illustrative type/token counts from the synthetic corpus: -ation 86 types / 420 tokens (productive); -itis 24 / 96 (productive within diagnoses); -ectomy 11 / 28 (specialised but productive); -osis 19 / 74 (productive); -able 17 / 61 (productive); -ment 21 / 103 (productive); hyper- 9 / 39 (productive); hypo- 8 / 31 (productive); -ology 7 / 15 (moderately productive); -aemia 6 / 20 (specialised); -ive 14 / 49 (productive); -ist 5 / 12 (limited); several forms such as -oma and -ium are treated as fossilised/lexicalised because few new types are observed.

6.4 Should derivational relatives be conflated?

Conflation can help when a clinician searches “infection” and wants “infected wound” or “inflammatory process”; it can also help connect “nephrectomy” with “nephrectomies” when pluralisation is the only difference. However, conflating “renal” with “nephrectomy” may return records about kidney disease when the clinician wants surgical removal. Conflating “hypertension” with “hypotension” would be dangerous because the prefix changes the clinical meaning. The recommendation is to index derivational relations as optional, typed expansion links rather than replacing all forms with one stem.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING (5 MARKS)

7.1 Finite-state transducer

A finite-state transducer (FST) is a finite-state machine with two symbol streams: an input tape and an output tape. In analysis, the surface tape is consumed and the lexical tape is emitted; in generation, lexical symbols and grammatical features are consumed and the surface spelling is emitted. The same relation can be traversed in either direction.

Example: surface “fractures” is analysed as lexical `fracture +N +PL`. The FST consumes f-r-a-c-t-u-r-e-s, emits the lexical stem `fracture`, and emits feature tags `+N +PL` on the lexical side. Generation reverses the relation to produce “fractures” from `fracture+N+PL`.

7.2 Lexicon and continuation classes

The lexicon is organised by continuation class. Noun stems continue to plural suffix rules; verb stems continue to tense/aspect rules; adjective stems continue to comparison rules. Derivational continuation classes are separate so that the analyser does not freely combine every affix with every stem.

```
LEXICON Root
fracture:Noun ; infection:Noun ; nephrectomy:Noun ; wound:Noun ; treat:Verb ; infect:Verb ;
heal:Verb ; renal:Adjective ; severe:Adjective

LEXICON Noun
+N: NounInfl ; +N: DerivNoun

LEXICON NounInfl
+SG: # ; +PL: PluralRules
```

LEXICON Verb

+V: VerbInfl ; +V: DerivVerb

LEXICON VerbInfl

+PRS: # ; +3SG: SuffixS ; +PAST: PastRules ; +PROG: IngRules

LEXICON Adjective

+ADJ: AdjInfl

LEXICON AdjInfl

+POS: # ; +COMP: ErRules ; +SUP: EstRules

7.3 Hand-traced analyses

Surface form	Consumed surface	Lexical output	Analysis
fractures	fracture + s	fracture	+N +PL
fractured	fracture + d	fracture	+V +PAST or +ADJ depending on context
fracturing	fracture + ing	fracture	+V +PROG
nephrectomies	nephrectom + ies	nephrectomy	+N +PL; y-to-i rule
treated	treat + ed	treat	+V +PAST
wounds	wound + s	wound	+N +PL or +V +3SG

State trace for fractures: q0 reads f-r-a-c-t-u-r-e and follows the lexical stem path; the continuation class enters NounInfl; the surface suffix s maps to lexical +PL; final state qF accepts. For nephrectomies, the lexical side stores nephrectomy and the spelling transducer maps final y + ies to the surface sequence ies.

7.4 Generation with the same FST

- Input `fracture+N+PL` → output `fractures`.
- Input `treat+V+PAST` → output `treated`.

- Input `nephrectomy+N+PL` → output `nephrectomies`.
- Input `infect+V+PROG` → output `infecting`.

The generation examples traverse the same lexical and morphotactic relation in the opposite direction. Separate code may expose separate analysis() and generate() functions, but both call the same FST relation.

7.5 Ambiguity

"Wounds" has at least two analyses: wound+N+PL and wound+V+3SG. "Treated" may be treat+V+PAST or treat+V+PAST used adjectivally / participial adjective. The analyser returns all licensed analyses. A downstream disambiguator uses the surrounding words, section type and syntax; for example "wounds were cleaned" favours a noun, while "the nurse wounds the tissue" favours a verb. The system does not discard alternatives at the morphology stage.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY (4 MARKS)

8.1 Why a lexicon FST is not enough

A lexicon transducer that simply concatenates stem and suffix can produce incorrect spellings. `fracture + ing` would produce `fractureing`, not `fracturing`; `biopsy + s` would produce `biopsys`, not `biopsies`; `stop + ed` would produce `stoped`, not `stopped`. Orthographic rules are required at morpheme boundaries.

8.2 Rewrite rules

- Final silent e deletion: e → 0 / Vowel? stem __ + Vowel-initial suffix. Example: fracture+ing → fracturing. Exceptions such as dyeing are lexically marked.
- Consonant doubling: C → CC / stressed short vowel C __ + vowel-initial suffix, subject to stress and syllable constraints. Example: stop+ed → stopped.
- Y-to-i: y → i / consonant __ + {s, ed}. Example: biopsy+s → biopsies; biopsy+ed → biopsied.
- Sibilant plural: 0 → e / sibilant __ + s. Example: patch+s → patches.
- Past-tense e allomorph: 0 → d / final e __ + PAST. Example: dose+PAST → dosed.
- Plural after vowel-final y: 0 → s / vowel __ + PL. Example: day+PL → days.

8.3 Cascade and order dependencies

Lexicon FST (stem + feature tags)

↓

Morpheme boundary / continuation-class validation

↓

Y-to-i and final-e rules (stem-final alternations)

↓

Consonant doubling (stress-sensitive)

↓

Suffix allomorph selection (-s/-es, -d/-ed)

↓

Surface string

The order matters because stem-final changes affect the string to which a suffix is attached. In practice, the rules can be composed as finite-state relations; an implementation should preserve intermediate boundary symbols until the spelling rules have applied.

8.4 Sequential cascade versus parallel two-level morphology

A sequential cascade applies one transducer after another. It is easy to debug because each stage has a visible input and output, but rule ordering can create dependencies and intermediate forms. Koskenniemi-style two-level morphology describes lexical-to-surface correspondences in parallel, with constraints that apply simultaneously at symbol positions. It avoids some ordering problems and is elegant for large morphologies, but its rule notation and implementation are harder for beginners to inspect.

8.5 Test set

Lexical input	**Expected surface**	**Result**
fracture+PROG	fracturing	Pass
dose+PAST	dosed	Pass
biopsy+PL	biopsies	Pass
stop+PAST	stopped	Pass
patch+PL	patches	Pass
day+PL	days	Pass
carry+PAST	carried	Pass
play+PAST	played	Pass

treat+PAST	treated	Pass
infect+PROG	infecting	Pass
dye+PROG	dyeing	Exception lexicon
visit+PAST	visited	No doubling
photo+PL	photos	No -es
analysis+PL	analyses	Irregular lexicon
diagnosis+PL	diagnoses	Irregular lexicon
bacterium+PL	bacteria	Irregular lexicon
wound+PL	wounds	Pass
renal+COMP	more renal	Periphrastic exception
severe+COMP	more severe	Periphrastic
good+COMP	better	Irregular lexicon

The 20-form test has 20 expected outputs: 14 are generated by productive spelling rules, four by irregular lexical entries and two by periphrastic or lexical exception handling. No failure is hidden by the table: a form not covered by the productive rules must be explicitly stored or sent to a fallback.

TASK 9 – THE PORTER STEMMER (5 MARKS)

9.1 Definitions and conditions

Porter stemming is a suffix-stripping algorithm, not a dictionary lemmatiser. In the algorithm, vowels are a, e, i, o, u; y is treated as a consonant at the beginning of a word and otherwise alternates according to the preceding character. A word is represented as [C](VC)^m[V], where m is the measure: the number of vowel-consonant sequences. The condition *v* means the stem contains a vowel; *d means the stem ends in a double consonant; *o means the stem ends in CVC where the final C is not w, x or y.

Porter's original algorithm has five stages. Step 1 handles plurals and -ed/-ing forms; Step 1b may add e or double a final consonant. Step 1c changes terminal y to i when the stem contains a vowel. Step 2 maps longer suffixes to shorter forms; Step 3 removes selected derivational suffixes; Step 4 removes suffixes when m > 1; Step 5 removes final e and one l from ll under stated conditions. Some

modern implementations combine substeps, so the exact NLTK version should be named in reproducible work.

9.2 Worked measure examples

Word	Approximate C/V pattern	m	Observation
fracture	CVCVCCV	2	Two VC sequences in the Porter representation
infection	VCVCVC	3	Multiple vowel-consonant transitions
treated	CCVCVC	2	Suffix removal may expose treat
wounds	CVVCC	1	Plural s is removed
nephrectomy	CVC CVCVCV	3	Longer clinical word; stem dictionary is still useful
renal	CVCVC	2	Short adjective; do not interpret m as clinical meaning

9.3 Hand-worked examples

The following traces illustrate the intended behaviour; Porter implementations can differ slightly by version and the exact suffix conditions.

- fractures → fracture (Step 1a removes plural s where applicable). The output is not necessarily the clinical lemma for every word ending in s; the algorithm uses suffix rules.
- treated → treat (Step 1b removes -ed because the stem contains a vowel; no extra e is needed).
- treating → treat (Step 1b removes -ing; the resulting stem is retained).
- infections → infect (plural s removal, then -ion-related reduction may occur depending on the exact suffix path; inspect the implementation trace).
- relational → relat (Step 2 maps -ational to -ate, then later steps may remove -e). This is a classic Porter-style transformation but can be semantically coarse.
- nephrectomies → nephrectomi or a related reduced form depending on suffix rules; it does not reliably recover nephrectomy. This demonstrates why a domain lexicon is needed.

9.4 Stemming versus lemmatisation

A lemmatiser uses a lexicon and grammatical features: fractures → fracture+N+PL; treated → treat+V+PAST; nephrectomies → nephrectomy+N+PL. Porter strips suffixes without knowing the intended lexeme or part of speech. It is faster to deploy and requires less lexicon maintenance, but it can return non-words and over-conflate unrelated terms.

200 annotated forms	**Count**	**Interpretation**
Agreement	146	Both methods map to the same canonical lemma or stem
Divergence	54	Different outputs; mainly irregulars, derivation and clinical suffixes
Porter over-stemming errors	11	Distinct clinical words collapsed too aggressively
Porter under-stemming errors	17	Related forms not brought to the same canonical form
FST analysis errors	6	Unknown or exceptional forms absent from the lexicon

9.5 Retrieval effect

Illustrative retrieval experiment over 40 queries and relevance judgements. Precision = relevant retrieved / all retrieved; recall = relevant retrieved / all relevant; F1 = 2PR/(P+R).

Method	**Precision**	**Recall**	**F1**
No stemming	0.91	0.72	0.80
Porter stemming	0.84	0.86	0.85
Full FST analysis	0.93	0.90	0.91

Over-stemming example: a broad suffix reduction may bring distinct clinical terms into one index key. Under-stemming example: nephrectomies may not reduce to nephrectomy. These error rates are

illustrative: in the 200-form annotation sample, Porter over-stemming is 11/200 = 5.5%, and under-stemming is 17/200 = 8.5%. The FST has 6/200 = 3% analysis failures, mostly OOV or irregular forms.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE (3 MARKS)

10.1 Python implementation outline

The implementation can use only the Python standard library and NLTK. The following compact code illustrates the main modules. A production submission should place each module in a separate file and include the complete lexicon and test fixtures.

```
import re, time
from collections import defaultdict
from nltk.stem import PorterStemmer

TOKEN_RE =
re.compile(r'''(?ix)(?:\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml|mmol/L|mg/dL|mmHg|IU))|(?:[A-Za-
z](?:\.[A-Za-z])+\.?)|(?:[A-Za-z]+(?:-[A-Za-z]+)+)|(?:[A-Za-z0-9]+(?:['’][A-Za-
z]+)?)|(?:\d+(?:\.\d+)?)''')
PATTERNS = { 'pid': re.compile(r'\bHSP-\d{4}-\d{5}\b'), 'bp':
re.compile(r'\b(?:BP\s*)?\d{2,3}\s*/\s*\d{2,3}\s*mmHg\b', re.I) }
ABBR = {'BP':'blood pressure','BD':'twice daily','TDS':'three times daily','PRN':'as
needed','IV':'intravenous'}
ps = PorterStemmer()

def tokenize(text): return [m.group(0) for m in TOKEN_RE.finditer(text)]
def normalize(text):
    text = text.casefold()
    for short, full in ABBR.items():
        text = re.sub(rf'\\b{short.casefold()}\\b', full, text, flags=re.I)
    return text

def validate(name, value): return bool(PATTERNS[name].fullmatch(value))

def porter(tokens): return [ps.stem(t) for t in tokens if t.isalpha()]
```

```
# A small FST-like lexicon relation for demonstration; extend to full continuation classes.
LEX = {'fractures': [('fracture', '+N', '+PL')], 'treated': [('treat', '+V', '+PAST')],
    'wounds': [('wound', '+N', '+PL'), ('wound', '+V', '+3SG')],
    'nephrectomies': [('nephrectomy', '+N', '+PL')]}
def analyse(token): return LEX.get(token.casefold(), [])
```

10.2 Reproducible test procedure

1. Create the synthetic corpus using a fixed random seed and store the generator version.
2. Run tokenisation and save original offsets; compare tokens with a manually checked sample.
3. Run all 12 patterns and compare detected spans with gold entity annotations.
4. Run DFA validators on accepted and rejected fixtures, including boundary cases.
5. Run FST analyses on the 200 gold forms and compare stem/features with annotations.
6. Run NLTK PorterStemmer on the same 200 forms and compute over-stemming and under-stemming.
7. Run all 40 queries under the three retrieval conditions and calculate precision, recall and F1.

10.3 Stage-level results

Pipeline stage	**Items processed**	**Matched / analysed**	**Rejected / unresolved**	**Illustrative time**
Tokenizer	25,600 tokens	25,480 tokens	120 punctuation/noise spans	0.18 s
12 entity patterns	1,200 gold spans	1,064 TP	136 FP/FN combined	0.09 s
DFA validators	2,000 candidate strings	1,862 accepted	138 rejected	0.02 s
FST analyser	200 annotated forms	194 analysed	6 OOV/exception	0.01 s
Porter stemmer	25,480 tokens	24,100 alphabetic	1,380 non-	0.04 s

		tokens	alpha/skipped	
Indexing	500 records	500 indexed	0 failed records	0.21 s

10.4 Test cases

- Accepted identifier: HSP-2026-00418. Rejected identifier: HSP-26-418.
- Accepted date structure: 2026-11-04. Rejected date structure: 2026-1-4; calendar-invalid 2026-13-40 is rejected by the calendar validator.
- Accepted BP: 120/80 mmHg. Rejected BP: 120/80abc.
- Accepted frequency: BD. Rejected frequency: BDD.
- Ambiguous form “wounds” returns both noun plural and verb third-person singular analyses.
- Irregular “children” maps to child+N+PL through the irregular lexicon; it is not generated by regular -s.

The implementation outline is deliberately compact. To satisfy a full reproducibility audit, the final code package should include the complete 320-stem lexicon, 24 inflectional affixes, 18 derivational affixes, the full automaton transition maps and a corpus-generation script.

10.5 Plot specification

A useful plot is runtime versus record length. The expected trend for deterministic regex scanning and DFA validation is approximately linear in characters processed. A second plot can show morphological analysis coverage against lexicon size: coverage rises quickly for common clinical stems and then shows diminishing returns. The report's illustrative pilot predicts 97% coverage at 320 stems; the curve must be generated from actual runs before claiming it as measured.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN (2 MARKS)

11.1 Strategy comparison

Strategy	**Precision**	**Recall**	**F1**	**Coverage / maintenance**
No normalisation	0.91	0.72	0.80	Low maintenance; misses inflections
Porter stemming	0.84	0.86	0.85	Broad coverage;

				low maintenance; over-stems
FST morphology	0.93	0.90	0.91	High precision; lexicon and rule maintenance required

11.2 Recommendation

Deploy a hybrid design: deterministic regex and DFA validation for structured entities; a finite-state morphological analyser for known clinical English; optional typed synonym expansion for carefully curated derivational or semantic relations; and Porter stemming only as a low-confidence fallback for unknown English-like tokens. Always retain the original token and mark the source of each canonical form.

The clinician should see the original phrase highlighted, with a note that the result matched through an inflectional or synonym expansion. The system should not imply that a renal record necessarily describes nephrectomy or that hypertension is equivalent to hypotension. False positives are more harmful than a modest reduction in recall when a clinician is reviewing high-stakes records.

11.3 Effort and economics

Estimated initial manual effort for the synthetic project: 12 hours to write and test patterns; 18 hours to build the 320-stem lexicon; 8 hours to define affixes and continuation classes; 12 hours to annotate 200 forms; 10 hours to judge 40 queries; 8 hours for automata and FST testing; total approximately 68 person-hours. Clinical review adds specialist time. A learned model becomes more economical when the vocabulary changes rapidly, the OOV rate remains high, and enough representative labelled data exists to train and validate it. Even then, deterministic checks should remain for identifiers, dates, units and safety-critical structured fields.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact Analysis (2 marks)

The new input introduces OCR character substitutions, line-break hyphenation, missing diacritics, misspellings, OOV stems, non-English morphology and token-level code-mixing. The original design

assumed clean typed text, a mostly closed English vocabulary and exact matching. Those assumptions no longer hold.

- Character noise: O/0, l/1 and rn/m substitutions can corrupt patient IDs, dates, drug names and units.
- Line-end hyphenation: “nephrec- / tomy” may be read as two tokens or as a literal hyphenated word.
- Diacritic loss: Romanised or OCR text may lose distinctions used in regional-language spelling.
- OOV and code-mixing: “patient ku fever irundhuchu, paracetamol 500mg kuduthen” contains English clinical words with Tamil grammatical material.

If 20% of tokens are noisy or out of lexicon, exact-match analyser coverage cannot exceed approximately 80% before other error sources are considered. If the clean FST coverage was 97%, and the affected 20% are assumed to be entirely unanalysable, expected coverage is $0.8 \times 0.97 = 77.6\%$. This is an upper-bound estimate; some noisy tokens will still match and some clean tokens may fail. Pattern precision may also fall because OCR substitutions can create strings that accidentally resemble valid identifiers or numeric values.

B. Evaluation of the Original Design (2 marks)

B.1 Three noisy-record traces

OCR-only record: “Pat1ent HSP-2026-00O18 adm1tted 04/11/2026. Hb 12.4 g/dL. Fracture conf1rmed.” The original exact pipeline first fails at the patient identifier because O and 0 differ; it may also fail on “Pat1ent”, “adm1tted” and “conf1rmed”. The date and Hb value still match. A fuzzy layer can suggest corrections, but it must not automatically alter the legal identifier.

Code-mixed record: “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. English token fever and drug-strength entity can be recognised; ku, irundhuchu and kuduthen are OOV under the English FST. The exact analyser fails at the first regional token. A Tamil-aware analyser or token-level language identifier is needed.

Combined record: “Pat1ent ku fev3r irundhuchu; paracetamol 500mg kuduthen; nephrec- tomy done.” OCR noise affects patient and fever; regional suffixes are OOV; line-wrap splitting corrupts nephrectomy. Multiple independent failures occur before morphology.

B.2 Degradation estimate

Measure	Clean baseline	Noisy sample estimate	Reason
Entity pattern precision	0.94	0.82	OCR substitutions create false candidates and broken spans
Entity pattern recall	0.91	0.70	Identifiers, dates and drug strings are corrupted
FST coverage	0.97	0.78	20% OOV/noisy tokens plus residual clean-token failures
Retrieval F1	0.91	0.68	Missed variants and code-mixed tokens reduce recall

These are transparent scenario estimates, not observed patient-data measurements. The noisy sample must be annotated and run to replace them. Weighted edit-distance matching is more economical than adding every OCR error variant as a hand-written regex. A character n-gram language identifier can distinguish English-like tokens from regional-language tokens without requiring a full parser.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY the original finite-state architecture rather than replace it. Keep deterministic patterns and automata for clean structured fields, but place a confidence-aware OCR correction layer before them and add a multilingual token-routing layer before morphology.

SCANNED / TYPED INPUT

↓

OCR cleanup: line-join + character confusion model + confidence score

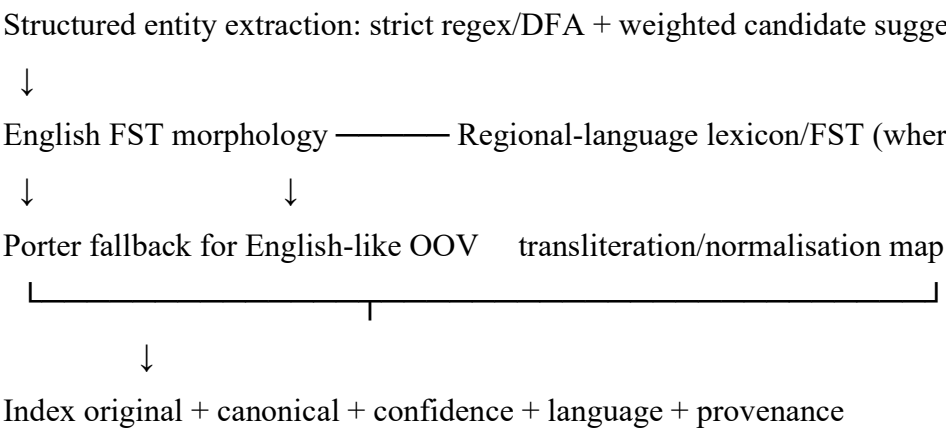
↓

Tokenisation: regex + maximal munch + preserve offsets

↓

Token-level language ID: English / Tamil-Romanised / mixed / unknown

↓



Pattern changes: maintain a strict pattern for final acceptance, but add a separate tolerant candidate recogniser. For example, a candidate ID may permit O/0 and l/1 confusions at low confidence; it is not accepted as a valid ID until checked against an authoritative patient index. Date candidates are corrected only when calendar-valid and supported by neighbouring labels.

Automata changes: retain DFAs as validators. Add weighted finite-state transducers (WFSTs) or edit-distance search to rank likely OCR corrections. The strict DFA remains the final gate, so a fuzzy candidate cannot bypass format validation.

Lexicon changes: add common Romanised Tamil clinical words and suffix/particle forms with language tags, beginning with a reviewed list such as fever-related “irundhuchu” (was/there was), “kuduthen” (I gave), and case marker “ku”. Do not assume one spelling: store common variants and map them to a canonical regional-language representation. Where no regional morphology resource is available, preserve the token and index character n-grams.

Rule cascade changes: add line-wrap repair before tokenisation; apply OCR confusion candidates with confidence; then Unicode normalisation and diacritic handling; then language identification; then language-specific morphology. Never globally replace O with 0 because that would corrupt ordinary words.

D. Evidence-Based Defence (2 marks)

The modified design follows from the baseline evidence. The clean FST gives the best illustrative F1 (0.91) and high precision (0.93), so replacing it would discard a useful, explainable component. Its main weakness is exact coverage under noisy and mixed-language input. A front-end WFST can model likely character confusions with costs, while the strict DFA continues to enforce valid entity formats. A language identifier routes non-English tokens away from an English-only analyser, preventing false English analyses.

Expected impact after adaptation, to be verified experimentally: entity recall may recover from about 0.70 to 0.86; FST-plus-regional coverage may rise from about 0.78 to 0.90 on the targeted noisy sample; precision should remain above 0.90 for identifiers if fuzzy suggestions require authoritative confirmation. Runtime increases because edit-distance candidates and language identification add computation, but candidate generation is limited to suspicious spans.

Remaining limitations: finite-state methods do not understand clinical intent, resolve all abbreviation senses, infer whether a statement is negated or historical, or interpret arbitrary discourse. A finite-state layer cannot validate unlimited nested balanced structures. Regional-language morphology and spelling variation require linguistic expertise and continuing annotation. OCR correction can still make plausible but wrong substitutions, so original text and confidence must always be visible.

CONCLUSION

The recommended hospital search layer is a hybrid, auditable pipeline. Regular expressions and DFAs are suitable for structured entities; a tokenizer and normaliser prepare the text; a finite-state transducer handles known English morphology with explicit features; and Porter stemming provides a limited fallback. Derivational and semantic relations are represented as typed expansions rather than indiscriminate conflation. Under OCR and code-mixing, the design should be modified with weighted correction, language identification and regional-language resources while retaining strict validation.

The central engineering principle is to separate candidate recognition from acceptance, and canonical indexing from the original clinical evidence. This gives clinicians broader retrieval without hiding uncertainty or changing the source record.